Comparación de Hill Climbing, sus variantes y Recocido Simulado en la función esfera

Angel Gabriel Aguilar Hernandez

Licenciatura en Ingeniería de Software, Universidad Veracruzana, México
zs24013324@estudiantes.uv.mx

Resumen. Se comparan cuatro algoritmos de búsqueda local, Hill Climbing (HC), Hill Climbing Steepest Ascent (SAHC), Steepest Ascent with Replacement (SAHC-R) y Recocido Simulado (SA), en la minimización de la función esfera con 10, 30, 50 y 100 dimensiones. Los parámetros de cada algoritmo se configuraron por dimensión mediante una búsqueda en rejilla con ejecuciones piloto y, con la configuración elegida, se hicieron 30 ejecuciones independientes con un presupuesto de $10{,}000 \cdot D$ evaluaciones y un umbral de éxito $f(\mathbf{x}) \le 10^{-8}$. HC y SAHC alcanzaron el umbral en todas las ejecuciones; SA lo logró en 30 de 30 con $D = 10$ y 15 de 30 con $D = 30$, y SAHC-R sólo en 9 de 30 con $D = 10$. HC fue el más rápido y el que menos evaluaciones usó hasta $D = 50$; SAHC usó menos evaluaciones con $D = 100$ y fue el más estable, con 100 % de éxito y la menor dispersión del esfuerzo en la mayoría de las dimensiones. En un problema unimodal, la capacidad de SA y SAHC-R para aceptar soluciones peores sólo consume evaluaciones.

Palabras clave: Búsqueda local · Hill Climbing · Recocido simulado · Función esfera · Configuración de parámetros.

1 Introducción

Los algoritmos de búsqueda local mejoran una sola solución candidata modificándola poco a poco: en cada iteración generan uno o varios vecinos con un operador de perturbación (*Tweak*) y deciden, con una regla de aceptación, cuál sustituye a la solución actual [1, 4]. Hill Climbing y el Recocido Simulado comparten ese esqueleto y difieren en dos decisiones: cuántos vecinos se examinan por iteración y si se permite pasar a una solución peor.

Este trabajo compara cuatro de estos algoritmos tal como los presenta Luke [1]: Hill Climbing (HC), Hill Climbing Steepest Ascent (SAHC), Hill Climbing Steepest Ascent with Replacement (SAHC-R) y Simulated Annealing o Recocido Simulado (SA) [2]. Los cuatro se aplican a la minimización de la función esfera con 10, 30, 50 y 100 dimensiones. Para cada algoritmo y dimensión se configuran los parámetros, se hacen 30 ejecuciones independientes y se reportan el mejor, el peor, la media, la mediana y la desviación estándar del valor final de la función objetivo, junto con el tiempo de ejecución y el número de evaluaciones. Con esos resultados se identifica el algoritmo más estable y se discuten las ventajas y desventajas de cada implementación.

2 Problema y algoritmos

2.1 Función esfera

La función esfera se define como

$$f(x) = \sum_{i=1}^{D} x_i^2, \quad x \in [-10,\ 10]^D \tag{1}$$

donde D es el número de dimensiones. Su mínimo global es $f(\mathbf{0}) = 0$. Es continua, convexa, separable y unimodal, de modo que no tiene óptimos locales que atrapen a la búsqueda: mide la capacidad de *explotación* de un algoritmo, es decir, qué tan rápido y con qué precisión se acerca al óptimo. Como los pseudocódigos de [1] maximizan una calidad, aquí se usa calidad = $-f$.

2.2 Operador de perturbación

Los cuatro algoritmos usan el mismo operador, la convolución uniforme acotada (Algoritmo 8 en [1]). Cada componente x_i se modifica con probabilidad p sumándole un ruido uniforme en $[-r, r]$; si el resultado sale del dominio, el ruido se vuelve a muestrear. La probabilidad se expresa como $p = k/D$, donde k es el número esperado de componentes modificadas, para que un mismo k produzca perturbaciones de tamaño comparable en cualquier dimensión. La solución inicial se genera de manera uniforme en el dominio.

2.3 Algoritmos comparados

Hill Climbing (HC). Algoritmo 4 de [1]. En cada iteración genera un vecino R de la solución actual S y lo acepta sólo si es estrictamente mejor. Usa una evaluación por iteración.

Steepest Ascent (SAHC). Algoritmo 5 de [1]. Genera n vecinos de S, toma el mejor y lo acepta sólo si mejora a S. Muestrear n vecinos aproxima la dirección de mayor descenso a cambio de n evaluaciones por iteración.

Steepest Ascent with Replacement (SAHC-R). Algoritmo 6 de [1]. Igual que SAHC, pero S se reemplaza siempre por el mejor de los n vecinos, aunque sea peor; la mejor solución encontrada se guarda aparte (*Best*). Así puede abandonar un óptimo local, pero también alejarse del óptimo global.

Simulated Annealing (SA). Algoritmo 13 de [1], propuesto por Kirkpatrick et al. [2]. Genera un vecino por iteración y lo acepta siempre que mejora; si empeora, lo acepta con probabilidad

$$P(\text{aceptar } R) = e^{\frac{f(S)-f(R)}{T}} \tag{2}$$

donde T es la temperatura. T parte de T_0 y se enfría geométricamente después de cada evaluación, de modo que llega a $T_f = 10^{-10}$ justo cuando se agota el presupuesto de B evaluaciones:

$$T_{t+1} = \alpha\, T_t, \qquad \alpha = \left(\frac{T_f}{T_0}\right)^{\frac{1}{B-1}} \tag{3}$$

Al igual que SAHC-R, conserva la mejor solución encontrada.

3 Diseño experimental

3.1 Presupuesto, criterio de paro y semillas

Se adoptó el criterio de la sesión especial de optimización con parámetros reales del CEC 2005 [3]: cada ejecución dispone de un máximo de $B = 10{,}000 \cdot D$ evaluaciones de la función objetivo (100,000, 300,000, 500,000 y 1,000,000 para D = 10, 30, 50 y 100) y termina antes si la mejor solución alcanza $f(\mathbf{x}) \le 10^{-8}$, valor a partir del cual el problema se considera resuelto (la condición «S es ideal» de los pseudocódigos de [1]). Se cuenta cada llamada a la función objetivo, incluida la de la solución inicial; en SAHC y SAHC-R el presupuesto puede agotarse a mitad de una iteración. Así, todos los algoritmos compiten con el mismo presupuesto y el número de evaluaciones refleja cuánto esfuerzo necesitó cada uno.

Por cada algoritmo y dimensión se hicieron 30 ejecuciones independientes con las semillas 1001 a 1030 del generador Mersenne Twister (mt19937) [5]. En la ejecución i los cuatro algoritmos usan la misma

semilla y, por lo tanto, parten de la misma solución inicial, de modo que las diferencias se deben al algoritmo y no al punto de partida.

3.2 Configuración de parámetros

Los parámetros se configuraron por separado para cada algoritmo y dimensión con una búsqueda en rejilla. Se probaron $k \in \{1, 2, 5, 10\}$ (es decir, $p = k/D$), $r \in \{0.001, 0.01, 0.1, 1\}$, $n \in \{2, 5, 10, 20, 50\}$ para SAHC y SAHC-R, y $T_0 \in \{0.001, 0.01, 0.1, 1, 10\}$ para SA: 16 configuraciones para HC y 80 para cada uno de los otros algoritmos. Cada configuración se ejecutó 5 veces con semillas distintas a las del experimento final (5001 a 5005), para no ajustar los parámetros a las mismas ejecuciones que después se reportan; en total fueron 5,120 ejecuciones piloto. Se eligió la configuración con más ejecuciones exitosas; en caso de empate, la de menor mediana de f (los valores menores que 10^{-8} cuentan como 10^{-8}); y si el empate persistía, la que usó menos evaluaciones en promedio. La Tabla 1 muestra la configuración elegida.

Tabla 1. Parámetros elegidos para cada algoritmo y dimensión. $p = k/D$: probabilidad de modificar cada componente (k entre paréntesis); r: amplitud del ruido; n: vecinos por iteración; T_0: temperatura inicial.

Algoritmo	Parámetro	$D = 10$	$D = 30$	$D = 50$	$D = 100$
Hill Climbing (HC)	p (k)	0.200 (2)	0.067 (2)	0.040 (2)	0.010 (1)
	r	0.01	0.01	0.01	0.01
Steepest Ascent (SAHC)	p (k)	0.200 (2)	0.067 (2)	0.040 (2)	0.020 (2)
	r	0.01	0.01	0.01	0.01
	n	2	5	2	2
Steepest Ascent with Replacement (SAHC-R)	p (k)	0.100 (1)	0.033 (1)	0.020 (1)	0.010 (1)
	r	0.1	0.1	0.1	0.1
	n	20	20	50	20
Simulated Annealing (SA)	p (k)	0.100 (1)	0.033 (1)	0.020 (1)	0.010 (1)
	r	0.01	0.01	0.01	0.01
	T_0	0.001	0.001	0.001	0.001

Se observan tres patrones. Primero, HC, SAHC y SA eligieron el mismo tamaño de paso, $r = 0.01$, y perturbaciones de una o dos componentes ($k \le 2$) en todas las dimensiones: en la esfera conviene dar pasos pequeños sobre pocas coordenadas, porque cada coordenada mejora casi de forma independiente (la función es separable). Con $r = 0.1$, HC no resolvió ninguna ejecución piloto en $D = 100$ (mediana de 1.57×10^{-7}). Segundo, SAHC prefirió pocos vecinos ($n = 2$ a 5): con presupuesto fijo, cada vecino adicional se paga con evaluaciones que ya no se usan en nuevas iteraciones; con $n = 20$ no resolvió ninguna ejecución piloto en $D = 100$. Tercero, SAHC-R necesitó lo contrario, $r = 0.1$ y $n = 20$ a 50, porque al moverse siempre requiere que el mejor de muchos vecinos sea casi siempre una mejora; con $r = 0.01$ su mejor mediana en $D = 100$ fue 6.58×10^{-7}, frente a 2.02×10^{-7} con $r = 0.1$. Por último, la mejor temperatura inicial de SA fue la menor de la rejilla, $T_0 = 0.001$; en $D = 100$ la mediana empeoró de 3.37×10^{-8} con $T_0 = 0.001$ a 4.86×10^{-8} con $T_0 = 10$. En una función unimodal, aceptar soluciones peores no aporta y SA funciona mejor cuanto más se parece a HC.

3.3 Medidas de desempeño y entorno

Para cada algoritmo y dimensión se reportan, sobre las 30 ejecuciones, el mejor, el peor, la media, la mediana y la desviación estándar muestral (con divisor 29) del valor de f de la mejor solución encontrada; el tiempo de CPU por ejecución y el número de evaluaciones de la función objetivo, ambos como media ± desviación estándar; y el número de ejecuciones exitosas. Los algoritmos y las estadísticas se implementaron desde cero en C++17, sin bibliotecas de optimización, y se compilaron con Apple clang 17 (-O2). Las ejecuciones finales se corrieron una tras otra en un Apple M2 con macOS; el tiempo se midió

con std::clock, que cuenta sólo el tiempo de CPU del proceso y no lo afecta que el sistema operativo atienda otros programas. El código fuente y los resultados completos en CSV acompañan este documento.

4 Resultados

Las Tablas 2 a 5 presentan los resultados para cada dimensión. Un valor de f menor que 10^{-8} indica que la ejecución alcanzó el umbral y se detuvo. Por eso, entre los algoritmos que siempre lo alcanzan, las diferencias en las filas de f no indican mayor precisión, sino dónde cayó la última mejora al cruzar el umbral; la comparación relevante entre ellos está en las evaluaciones y el tiempo.

Tabla 2. Resultados de 30 ejecuciones con $D = 10$ (presupuesto: 100,000 evaluaciones). Las filas de Mejor a Desviación Estándar se refieren al valor de f de la mejor solución; el tiempo de CPU y las evaluaciones se dan como media ± desviación estándar. Una ejecución es exitosa si alcanza $f \leq 10^{-8}$.

	HillClimbing	HillClimbing Steepest Ascent	HillClimbing Steepest Ascent with Replacement	Simulated Annealing
Mejor	3.60×10^{-9}	4.79×10^{-9}	5.81×10^{-9}	6.13×10^{-9}
Peor	9.76×10^{-9}	1.00×10^{-8}	4.97×10^{-8}	9.97×10^{-9}
Media	7.74×10^{-9}	8.01×10^{-9}	1.93×10^{-8}	8.72×10^{-9}
Mediana	8.19×10^{-9}	8.21×10^{-9}	1.49×10^{-8}	8.73×10^{-9}
Desviación Estándar	1.69×10^{-9}	1.49×10^{-9}	1.25×10^{-8}	1.01×10^{-9}
Tiempo de ejecución (ms)	5.7 ± 0.7	5.8 ± 0.8	10.4 ± 0.9	9.2 ± 0.7
Número de evaluaciones de la función objetivo	36,454 ± 5,029	37,704 ± 4,768	95,958 ± 7,913	82,936 ± 4,828
Ejecuciones exitosas	30/30	30/30	9/30	30/30

Tabla 3. Resultados de 30 ejecuciones con $D = 30$ (presupuesto: 300,000 evaluaciones). Las filas de Mejor a Desviación Estándar se refieren al valor de f de la mejor solución; el tiempo de CPU y las evaluaciones se dan como media ± desviación estándar. Una ejecución es exitosa si alcanza $f \leq 10^{-8}$.

	HillClimbing	HillClimbing Steepest Ascent	HillClimbing Steepest Ascent with Replacement	Simulated Annealing
Mejor	3.18×10^{-9}	5.29×10^{-9}	1.73×10^{-8}	7.93×10^{-9}
Peor	9.95×10^{-9}	9.90×10^{-9}	1.23×10^{-7}	2.08×10^{-8}
Media	8.85×10^{-9}	8.60×10^{-9}	6.69×10^{-8}	1.14×10^{-8}
Mediana	9.33×10^{-9}	8.90×10^{-9}	6.34×10^{-8}	1.00×10^{-8}
Desviación Estándar	1.39×10^{-9}	1.21×10^{-9}	2.79×10^{-8}	2.91×10^{-9}
Tiempo de ejecución (ms)	39.5 ± 5.4	52.5 ± 6.1	72.9 ± 6.7	75.2 ± 3.3
Número de evaluaciones de la función objetivo	151,255 ± 19,490	178,258 ± 12,705	300,000 ± 0	296,577 ± 5,475
Ejecuciones exitosas	30/30	30/30	0/30	15/30

Tabla 4. Resultados de 30 ejecuciones con $D = 50$ (presupuesto: 500,000 evaluaciones). Las filas de Mejor a Desviación Estándar se refieren al valor de f de la mejor solución; el tiempo de CPU y las evaluaciones se dan como media ± desviación estándar. Una ejecución es exitosa si alcanza $f \leq 10^{-8}$.

	HillClimbing	HillClimbing Steepest Ascent	HillClimbing Steepest Ascent with Replacement	Simulated Annealing
Mejor	5.11×10^{-9}	5.21×10^{-9}	3.36×10^{-8}	1.17×10^{-8}
Peor	1.00×10^{-8}	9.98×10^{-9}	1.63×10^{-7}	2.28×10^{-8}
Media	9.15×10^{-9}	9.08×10^{-9}	1.11×10^{-7}	1.68×10^{-8}
Mediana	9.41×10^{-9}	9.52×10^{-9}	1.10×10^{-7}	1.62×10^{-8}
Desviación Estándar	1.10×10^{-9}	1.15×10^{-9}	3.01×10^{-8}	3.41×10^{-9}
Tiempo de ejecución (ms)	115.8 ± 12.5	125.3 ± 13.5	186.6 ± 10.4	193.4 ± 6.7
Número de evaluaciones de la función objetivo	292,321 ± 29,335	311,808 ± 24,462	500,000 ± 0	500,000 ± 0
Ejecuciones exitosas	30/30	30/30	0/30	0/30

Tabla 5. Resultados de 30 ejecuciones con $D = 100$ (presupuesto: 1,000,000 evaluaciones). Las filas de Mejor a Desviación Estándar se refieren al valor de f de la mejor solución; el tiempo de CPU y las evaluaciones se dan como media ± desviación estándar. Una ejecución es exitosa si alcanza $f \leq 10^{-8}$.

	HillClimbing	HillClimbing Steepest Ascent	HillClimbing Steepest Ascent with Replacement	Simulated Annealing
Mejor	8.91×10^{-9}	8.35×10^{-9}	1.13×10^{-7}	2.59×10^{-8}
Peor	9.97×10^{-9}	1.00×10^{-8}	3.90×10^{-7}	4.70×10^{-8}
Media	9.69×10^{-9}	9.68×10^{-9}	2.28×10^{-7}	3.41×10^{-8}
Mediana	9.78×10^{-9}	9.89×10^{-9}	2.11×10^{-7}	3.30×10^{-8}
Desviación Estándar	2.99×10^{-10}	4.02×10^{-10}	6.88×10^{-8}	4.96×10^{-9}
Tiempo de ejecución (ms)	550.4 ± 32.9	563.0 ± 48.8	696.9 ± 32.5	753.6 ± 27.1
Número de evaluaciones de la función objetivo	801,472 ± 47,596	772,049 ± 60,012	1,000,000 ± 0	1,000,000 ± 0
Ejecuciones exitosas	30/30	30/30	0/30	0/30

HC y SAHC alcanzaron el umbral en sus 120 ejecuciones (30 por dimensión). HC fue el que menos evaluaciones necesitó con $D = 10$, 30 y 50 (36,454, 151,255 y 292,321 en promedio), mientras que con $D = 100$ SAHC lo superó (772,049 frente a 801,472). En tiempo de CPU, HC fue el más rápido en las cuatro dimensiones; con $D = 100$ aventajó a SAHC por poco (550.4 frente a 563.0 ms), porque cada evaluación de SAHC costó un poco más (729 frente a 687 ns en promedio). SA resolvió las 30 ejecuciones con $D = 10$, pero usando en promedio 82.9 % del presupuesto; con $D = 30$ resolvió 15 de 30 y con $D = 50$ y 100 ninguna, aunque quedó cerca del umbral (medianas de 1.62×10^{-8} y 3.30×10^{-8}). SAHC-R sólo resolvió 9 de 30 ejecuciones con $D = 10$ y ninguna en las demás dimensiones, con las peores medianas en todos los casos (hasta 2.11×10^{-7} con $D = 100$).

La Fig. 1 muestra la evolución de la mejor solución. Las curvas tienen dos fases. Mientras $|x_i|$ es mucho mayor que r, cada mejora acerca la componente al óptimo en una cantidad del orden de r, y en escala logarítmica eso se ve como una caída abrupta. Cuando $|x_i|$ se vuelve comparable con r, sólo los ruidos pequeños mejoran, la probabilidad de aceptar un vecino disminuye y el avance se vuelve lento: como el paso no se adapta, HC y SAHC dedican cerca de la mitad de sus evaluaciones a este refinamiento final. SAHC-R es el más rápido en la primera fase, gracias a su paso diez veces mayor y a que elige el mejor de

20 a 50 vecinos, pero se estanca por encima del umbral (medianas finales de 1.49×10^{-8} a 2.11×10^{-7}), porque al pasar siempre al mejor vecino oscila alrededor del óptimo a la escala de r.

En $D = 100$ la curva de SA coincide con la de HC (usan los mismos p y r) hasta cerca de 4×10^{5} evaluaciones y después se separa. La explicación es la distribución de equilibrio del recocido: a temperatura T, SA visita las soluciones con probabilidad proporcional a $e^{-f(x)/T}$, que en la esfera es una normal con varianza $T/2$ en cada componente, de modo que

$$E[f] = \sum_{i=1}^{D} E[x_i^2] = \frac{DT}{2} \tag{4}$$

Mientras f está muy por encima de $DT/2$, los empeoramientos que SA acepta son pequeños frente a las mejoras y avanza prácticamente como HC; al llegar a ese nivel se estanca y sólo baja al ritmo en que se enfría. Para que $DT/2 \le 10^{-8}$ hace falta $T \le 2\times10^{-8}/D$, temperatura que el enfriamiento de la Ec. (3) alcanza a 81.4 % del presupuesto con $D = 10$ (SA usó 82.9 %), a 88.2 % con $D = 30$ y a 95.7 % con $D = 100$, sin dejar margen para el refinamiento. Esto explica por qué SA deja de alcanzar el umbral al crecer D aunque su T_0 sea pequeña.

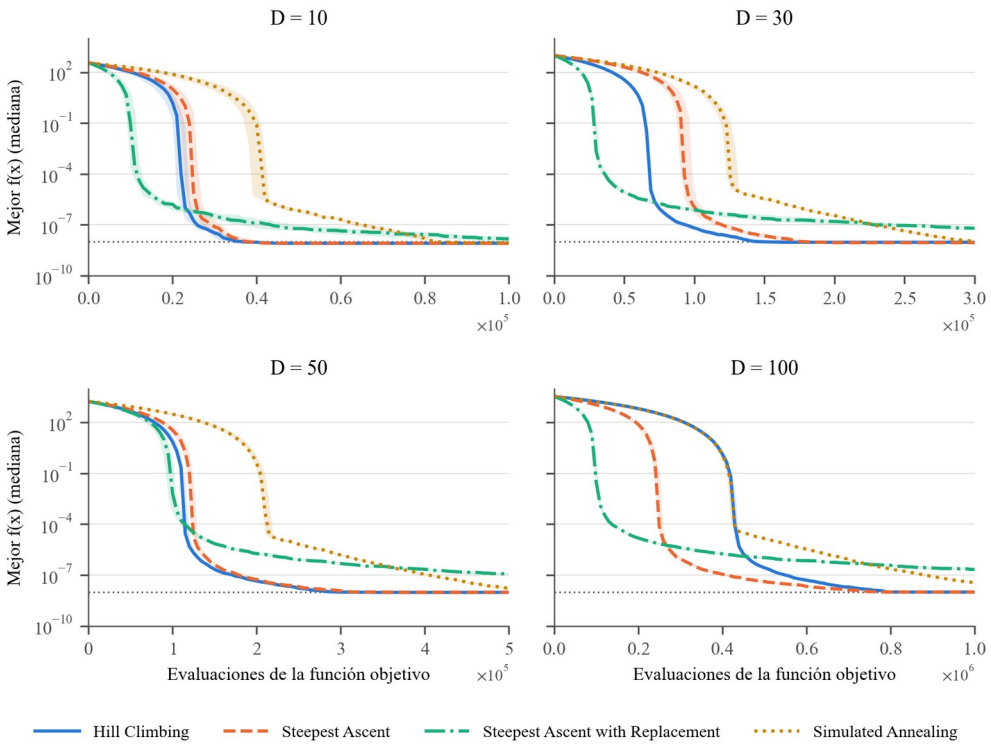


Fig. 1. Convergencia de los cuatro algoritmos: mediana (línea) y rango intercuartil (sombra) de la mejor $f(\mathbf{x})$ encontrada en las 30 ejecuciones, en escala logarítmica. La línea punteada marca el umbral de éxito 10^{-8}.

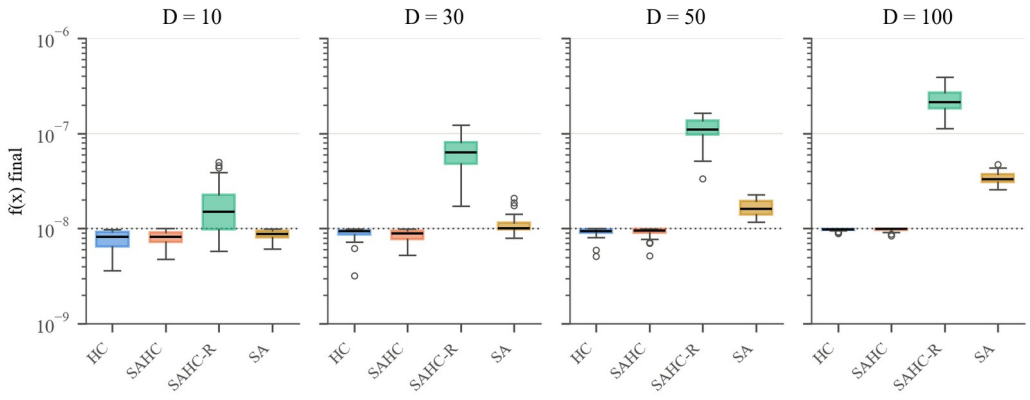


Fig. 2. Distribución del valor final de $f(\mathbf{x})$ en las 30 ejecuciones (escala logarítmica). La línea punteada marca el umbral 10^{-8}.

La Fig. 2 resume la dispersión del valor final. HC y SAHC quedan siempre justo por debajo del umbral. SA queda por encima a partir de $D = 30$, pero con cajas estrechas, y SAHC-R tiene las cajas más altas y más anchas en todas las dimensiones.

5 Discusión

5.1 ¿Qué algoritmo fue más estable?

Se considera más estable el algoritmo que produce resultados parecidos en todas las ejecuciones. Para los algoritmos que alcanzan el umbral, eso se mide con la tasa de éxito y la dispersión del esfuerzo que necesitan para alcanzarlo (el coeficiente de variación, CV = desviación estándar / media, de las evaluaciones); para los que no lo alcanzan, con el CV del valor final de *f*.

Con ese criterio, **Hill Climbing Steepest Ascent fue el algoritmo más estable**. Junto con HC es el único que resolvió el 100 % de las ejecuciones en las cuatro dimensiones, y entre los dos fue el de menor dispersión en el número de evaluaciones con $D = 10$, 30 y 50 (CV de 12.6 %, 7.1 % y 7.8 %, frente a 13.8 %, 12.9 % y 10.0 % de HC); con $D = 100$ HC fue algo más regular (5.9 % contra 7.8 %). La razón es que SAHC se queda con el mejor de *n* vecinos, lo que vuelve más regular el avance de cada iteración que en HC, donde depende de un solo vecino. HC queda en segundo lugar, muy cerca.

SA fue el más regular con $D = 10$ (CV de 5.8 % en las evaluaciones), pero por otra razón: lo que decide cuándo converge es el programa de enfriamiento, que es igual en todas las ejecuciones (Ec. 4). Esa regularidad no se traduce en éxito al crecer *D*: resolvió 15 de 30 ejecuciones con $D = 30$ y ninguna con $D = 50$ y 100, aunque con poca dispersión (CV de *f* de 20.3 % y 14.5 %). SAHC-R fue el menos estable: tuvo el mayor CV de *f* en todas las dimensiones (entre 27.2 % y 64.8 %) y la menor tasa de éxito.

5.2 Ventajas y desventajas

Hill Climbing. *Ventajas:* es el más simple (sólo *p* y *r*) y, en este problema, el más eficiente: usa una evaluación por iteración, resolvió todas las ejecuciones, fue el más rápido en tiempo de CPU en las cuatro dimensiones y necesitó la menor cantidad de evaluaciones con $D = 10$, 30 y 50. *Desventajas:* sólo acepta mejoras, así que en una función multimodal se detendría en el primer óptimo local que encuentre, algo que la esfera no pone a prueba. Además depende mucho del tamaño de paso: con $r = 0.1$ no resolvió ninguna ejecución piloto en $D = 100$. Y como *r* es fijo, gasta cerca de la mitad del presupuesto en el refinamiento final (Fig. 1).

Hill Climbing Steepest Ascent. *Ventajas:* tomar el mejor de *n* vecinos hace más regular el avance, lo que lo volvió el más estable, y con $D = 100$ fue el que menos evaluaciones necesitó (772,049 frente a 801,472

de HC). *Desventajas:* añade el parámetro n y, con presupuesto fijo, es sensible a él, porque cada vecino adicional cuesta una evaluación (Sección 3.2), y el valor elegido (n = 2 a 5) lo deja muy cerca de HC. Cada evaluación le costó algo más de tiempo que a HC, de modo que su ahorro de evaluaciones con D = 100 no se tradujo en menor tiempo. Comparte con HC la incapacidad de escapar de óptimos locales.

Hill Climbing Steepest Ascent with Replacement. *Ventajas:* es la única variante de Hill Climbing que puede empeorar a propósito, lo que en funciones multimodales le permitiría salir de óptimos locales, y guardar *Best* garantiza no perder la mejor solución. En la esfera fue el que más rápido redujo f al principio (Fig. 1). *Desventajas:* en un problema unimodal moverse siempre es contraproducente: cerca del óptimo, el mejor de los n vecinos suele ser peor que S y la búsqueda oscila a la escala de r, por lo que nunca alcanzó 10^{-8} con $D \geq 30$ y fue el menos estable. Además exige ajustar r y n en conjunto y necesitó muchos vecinos por iteración (n = 20 a 50).

Simulated Annealing. *Ventajas:* aceptar empeoramientos con una probabilidad que baja con la temperatura le permite explorar al inicio y explotar al final [2], que es lo que se necesita en funciones multimodales; usa una evaluación por iteración y con D = 10 fue el de comportamiento más regular. *Desventajas:* es el que más parámetros requiere (p, r, T_0, T_f y el programa de enfriamiento) y su desempeño depende de que la temperatura corresponda a la escala de f. En la esfera la exploración no aporta: la mejor T_0 fue la menor probada y, aun así, por la Ec. (4) sólo puede alcanzar 10^{-8} cuando la temperatura baja de $2\times10^{-8}/D$, lo que ocurre cada vez más cerca del final del presupuesto conforme crece D. Además, como no se detiene antes, consumió todo el presupuesto con $D \geq 50$ y fue el más lento con D = 30, 50 y 100.

6 Conclusiones

En la función esfera, los algoritmos que sólo aceptan mejoras fueron claramente superiores: HC y SAHC resolvieron todas sus ejecuciones en las cuatro dimensiones. HC fue el más rápido en todas ellas y el que menos evaluaciones usó hasta D = 50; SAHC usó menos evaluaciones con D = 100 y resultó el más estable. Las dos formas de exploración, moverse siempre (SAHC-R) y aceptar empeoramientos según la temperatura (SA), consumieron evaluaciones sin beneficio en un problema sin óptimos locales: SA sólo alcanzó el umbral cuando su temperatura bajó lo suficiente y SAHC-R quedó oscilando cerca del óptimo.

El experimento también muestra que la configuración de parámetros, sobre todo el tamaño de paso r, puede pesar tanto como la elección del algoritmo: con un r inadecuado incluso HC falla. Como trabajo futuro se propone repetir la comparación en funciones multimodales, como Rastrigin, donde la capacidad de SA y SAHC-R para escapar de óptimos locales sí podría compensar su costo, y probar un tamaño de paso que se adapte durante la búsqueda.

Uso responsable de la inteligencia artificial. Se utilizó inteligencia artificial como apoyo en tres actividades: investigación, completación de código y elaboración del documento. Las decisiones son del autor, quien revisó y validó todo el contenido.

Referencias

1. Luke, S.: Essentials of Metaheuristics. 2nd edn. Lulu (2013), https://cs.gmu.edu/~sean/book/metaheuristics/
2. Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P.: Optimization by simulated annealing. Science **220**(4598), 671–680 (1983)
3. Suganthan, P.N., Hansen, N., Liang, J.J., Deb, K., Chen, Y.P., Auger, A., Tiwari, S.: Problem definitions and evaluation criteria for the CEC 2005 special session on real-parameter optimization. Technical Report, Nanyang Technological University, Singapore (2005)
4. Russell, S., Norvig, P.: Artificial Intelligence: A Modern Approach. 4th edn. Pearson, Hoboken (2020)
5. Matsumoto, M., Nishimura, T.: Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. ACM Trans. Model. Comput. Simul. **8**(1), 3–30 (1998)